

Stack Implementation using Linked List in C

Source Code

```
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *top = NULL;

bool isEmpty();

// Function to create a new node
struct node* createNode() {
    int value;
    printf("Enter element: ");
    scanf("%d", &value);

    struct node* newNode = (struct
node*)malloc(sizeof(struct node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}

// Push operation
void push(){
    struct node *newNode = createNode();
    if(isEmpty()){
```

```

        top = newNode;
    }
    else{
        newNode->next = top;
        top = newNode;
    }
    printf("\nElement pushed");
}

// Pop operation
int pop(){
    struct node* temp = NULL;
    if(isEmpty()){
        printf("\nStack Underflow");
    }
    else{
        int ele = top->data;
        temp = top;
        top = top->next;
        free(temp);
        printf("\n%d popped.",ele);
    }
}

// Check if stack is empty
bool isEmpty(){
    return (top == NULL);
}

// Display stack elements
void display(){
    if(isEmpty()){
        printf("\nStack is empty.");
    }
    else{
        printf("\n");
        struct node *temp = top;
        while(temp != NULL){
            printf("%d\n",temp->data);
            temp = temp->next;
        }
    }
}

```

```

    }
}
// Main function - Menu Driven Program
int main() {
    int choice;
    int item;

    while (1) {
        printf("\n===== STACK MENU =====\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Check if Empty\n");
        printf("4. Display\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                push();
                break;
            case 2:
                item = pop();
                break;
            case 3:
                if (isEmpty())
                    printf("Stack is EMPTY!\n");
                else
                    printf("Stack is NOT empty.\n");
                break;
            case 4:
                display();
                break;
            case 5:
                printf("Exiting...\n");
                exit(0);
            default:
                printf("Invalid choice! Try again.\n");
        }
    }
    return 0;
}

```

Output :

===== STACK MENU =====

1. Push
2. Pop
3. Check if Empty
4. Display
5. Exit

Enter your choice: 1

Enter element: 10

Element pushed

===== STACK MENU =====

1. Push
2. Pop
3. Check if Empty
4. Display
5. Exit

Enter your choice: 1

Enter element: 20

Element pushed

===== STACK MENU =====

1. Push
2. Pop
3. Check if Empty
4. Display
5. Exit

Enter your choice: 1

Enter element: 30

===== STACK MENU =====

1. Push
2. Pop
3. Check if Empty
4. Display
5. Exit

Enter your choice: 1

Enter element: 40

Element pushed

===== STACK MENU =====

1. Push
2. Pop
3. Check if Empty
4. Display
5. Exit

Enter your choice: 4

40

30

20

10

===== STACK MENU =====

1. Push
2. Pop
3. Check if Empty

```
4. Display
5. Exit
Enter your choice: 2

40 popped.
===== STACK MENU =====
1. Push
2. Pop
3. Check if Empty
4. Display
5. Exit
Enter your choice: 4

30
20
10

===== STACK MENU =====
1. Push
2. Pop
3. Check if Empty
4. Display
5. Exit
Enter your choice: 2

30 popped.
===== STACK MENU =====
1. Push
2. Pop
```

```
3. Check if Empty
4. Display
5. Exit
Enter your choice: 4

20
10

===== STACK MENU =====
1. Push
2. Pop
3. Check if Empty
4. Display
5. Exit
Enter your choice: 5
Exiting...
```